

Knapsack Problem

Prerequisite knowledge: [Introduction to Dynamic Programming](#)

Introduction

Consider the following example:

[USACO07 Dec] Charm Bracelet

There are n distinct items and a knapsack of capacity W . Each item has 2 attributes, weight (w_i) and value (v_i). You have to select a subset of items to put into the knapsack such that the total weight does not exceed the capacity W and the total value is maximized.

In the example above, each object has only two possible states (taken or not taken), corresponding to binary 0 and 1. Thus, this type of problem is called "0-1 knapsack problem".

0-1 Knapsack

Explanation

In the example above, the input to the problem is the following: the weight of i^{th} item w_i , the value of i^{th} item v_i , and the total capacity of the knapsack W .

Let $f_{i,j}$ be the dynamic programming state holding the maximum total value the knapsack can carry with capacity j , when only the first i items are considered.

Assuming that all states of the first $i-1$ items have been processed, what are the options for the i^{th} item?

- When it is not put into the knapsack, the remaining capacity remains unchanged and total value does not change. Therefore, the maximum value in this case is $f_{i-1,j}$
- When it is put into the knapsack, the remaining capacity decreases by w_i and the total value increases by v_i , so the maximum value in this case is $f_{i-1,j-w_i} + v_i$

From this we can derive the dp transition equation: